Vorkurs Mathematik
-
Introductory Course Mathematics

Part 0 (organization, introduction)

Michael Luttenberger

Technische Universität München
School of Computation, Information and Technology
Department of Computer Science

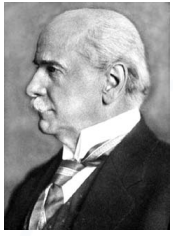
2026/09/07

For personal use only, no commercial use – TUM CIT Department of Computer Science

Overview

- Goals:
  - Recap of math (rules) taught at school using the terminology used at university.
  - "Warm up" for the actual (90min) lectures.
  - Socialize in the tutorials (talk to each other, make fun of the lecturer ...).
  - How to learn at university; what is expected from you in the tutorials.
- Organization:
  - Math lectures (TUE,THU): 08,10,15,17,22,24.
    Python tutorials (MON): 14,21, (TBD: 28) (e.g. use colab or jupyter-lab).
    Each approx. 2h (including breaks).
    In English as for both Garching and Heilbronn.
  - Tutorials both on-site (registration required) and online (DE/EN).
  - Questions during lecture via tweeedback. (Feel free to ask in German; new link at start of each lecture.)
  - Schedule, lecture material, ... via moodle.
  - In case of problems: mathe-vorkurs26@cit.tum.de

For personal use only, no commercial use – TUM CIT Department of Computer Science



[img src]

Walther Franz Anton von Dyck (6 December 1856 – 5 November 1934) [...] is credited with being the first to define a mathematical group, in the modern sense [...]. He laid the foundations of combinatorial group theory, being the first to systematically study a group by generators and relations.
Von Dyck was a student of Felix Klein [...]. He promoted technological education as rector of the Technische Hochschule of Munich.[...]
The Dyck language in formal language theory is named after him, as are Dyck's theorem and Dyck's surface in the theory of surfaces, together with the von Dyck groups, the Dyck tessellations, Dyck paths, and the Dyck graph. ([wikipedia])

- https://www.moodle.tum.de/course/view.php?id=3980772
- Password: aabxxabyababybxy

For personal use only, no commercial use – TUM CIT Department of Computer Science

Logic and mathematics

- Logic: from $\lambda o \gamma \iota \kappa \eta \ \tau \epsilon \chi \nu \eta$ ("art of reasoning"; not just "argument(ation)/reasoning")

  Study of the validity of arguments, how to deduce/infer, based on the definitions of if-then, and, or, not, all, exists, there is, ...

  e.g. if (1) all pigs can fly, and (2) all things that can fly are made of wood,
  then (necessarily) all pigs are made of wood.

- Mathematics: from $\mu \alpha \theta \eta \mu \alpha$ ("that which is learnt", "knowledge", also "science")

  Builds up on logic (sometimes the same), originally used as "precise language for describing nature", today the study of the concepts used originally for describing nature and rules for computing/calculating with these concepts:

  e.g. if $F = mg$, $F = ma$, $a = \dot{v}$, $v = \dot{s}$, $v(t_0) = 0$, $s(t_0) = 0$, and $g \in \mathbb{R}$ (Newton),
  then $v(t) = v(t_0) + \int_{t_0}^{t} g d\tau = gt$ and $s(t) = s(t_0) + \int_{t_0}^{t} v(\tau) d\tau = 1/2gt^2$.
  by virtue of the fundamental theorem of calculus w.r.t. the definitions of $\mathbb{R}$, "continuity", "differentiability", "integral", ... (the "actual" math).

- Computer science: ("compute", "calculate", "to reckon", "count", "tell")

  Can we build machines to apply the rules correctly (e.g. to predict the motion of planets)? Which rules do we need to implement? What can these machines do?

  ("*Pascal was led to develop a calculator by the laborious arithmetical calculations ...*")

For personal use only, no commercial use — TUM CIT Department of Computer Science

The work of Gottfried Wilhelm Leibniz (told by ChatGPT ...)



Indignum enim est excellentium virorum horas servili calculandi labore perire, qui Machina adhibita vilissimo cuique secure transcribi posset. (source)

You should still be able to validate the results produced by the machine (not only in case of LLMs): try to spot the typos, inconsistencies, etc and how they change ("bug fixing" starts with "Where is Waldo?").

For personal use only, no commercial use – TUM CIT Department of Computer Science

The work of Gottfried Wilhelm Leibniz (told by ChatGPT ...)



Indignum enim est excellentium virorum horas servili calculandi labore perire, qui Machina adhibita vilissimo cuique secure transcribi posset. (source)

You should still be able to validate the results produced by the machine (not only in case of LLMs): try to spot the typos, inconsistencies, etc and how they change ("bug fixing" starts with "Where is Waldo?").

For personal use only, no commercial use – TUM CIT Department of Computer Science

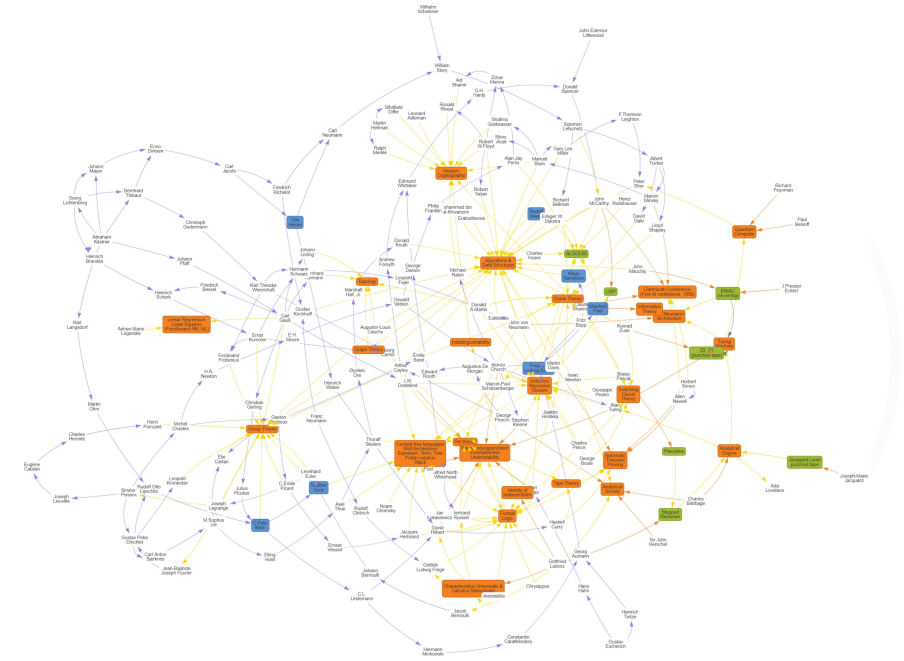
The work of Gottfried Wilhelm Leibniz (told by ChatGPT ...)



Indignum enim est excellentium virorum horas servili calculandi labore perire, qui Machina adhibita vilissimo cuique secure transcribi posset. (source)

You should still be able to validate the results produced by the machine (not only in case of LLMs): try to spot the typos, inconsistencies, etc and how they change ("bug fixing" starts with "Where is Waldo?").

For personal use only, no commercial use – TUM CIT Department of Computer Science



A bit of history:

- Orange node: “concept/idea”
- Green node: “hard-/software”
- Blue node: “professor at TUM”
- Yellow edge: “influenced”
- Red edge: “invented/created”
- Blue edge: “advised” (e.g. doctoral advisor)
- ⚠ Certainly incomplete and simplified, just an example of where logic (predicates), mathematics (relations, set theory, graph theory) and applied computer science (databases, AI, knowledge graphs) meet.
- ⚠ “Colors” serve in mathematics as abstraction (what is left after removing all unncessary information, reduced to the necessary information); the more colors, the more we can distinguish/separate, the finer the abstraction becomes.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Calculation, computation, algorithm

- One part of mathematics is deriving useful rules for computation/calculation and proving their validity/correctness.
  - A calculus: collection of rules for computing with certain objects.
  - In English, "calculus" is often used specifically for "infinitesimal calculus" referring to the rules derived by Leibniz resp. Newton for computing/calculating with integrals and derivatives.
  - See also calculus (disambiguation) on wikipedia.
  - "Calculus" actually means "small stone containing chalk", used as counter in an abacus (probably also a tally list).
  - Wiktionary considers "to calculate", "to compute" and also "to reckon" to be synonyms.

    All refer to the concept of counting, i.e. the natural numbers (cf. Peano axioms later).

For personal use only, no commercial use – TUM CIT Department of Computer Science

Calculation, computation, algorithm

- One part of mathematics is deriving useful rules for computation/calculation and proving their validity/correctness.
- A calculus gives rise to a (nondeterministic) algorithm:

  Given an initial "term" (depending on context also "expression", "formula") use the rules to rewrite the current term into another (simpler) term **until** some particular simple normal form (final result) is obtained.

  - to rewrite: "Clear blackboard and write simplified term" (cf.: "ritzen").
  - Determinism: at most one rule applicable at any time (a plan, a strategy).
  - Nondeterminism to determinism: try all applicable rules and remember all intermediate results (not very efficient, but sometimes the best we can do).
- Examples: To compute $a + b$ ($a, b \in \mathbb{N}_0$), while $a > 0$ and $b > 0$:

  v1: increment either a or b, and decrement the other.

  v2: increment a and decrement b.

  v3: if $a < b$, swap a and b; increment a and decrement b.

  Finally, rewrite $a + 0$ to a and $0 + b$ to b.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Calculation, computation, algorithm

- One part of mathematics is deriving useful rules for computation/calculation and proving their validity/correctness.
- Examples: To compute $a + b$ ($a, b \in \mathbb{N}_0$), while $a > 0$ and $b > 0$:
  - v1: increment either a or b, and decrement the other.
  - v2: increment a and decrement b.
  - v3: if $a < b$, swap a and b; increment a and decrement b.

  Finally, rewrite $a + 0$ to a and $0 + b$ to b.

$$0+4 \qquad \underline{1+3} \qquad 2+2 \qquad 3+1 \qquad 4+0$$

$$4$$

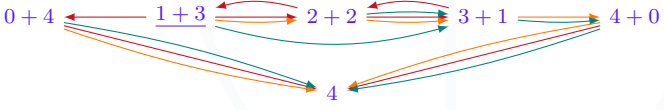
(An LLM should be able to translate v3 into actual (e.g.) Python code.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

Calculation, computation, algorithm

- One part of mathematics is deriving useful rules for computation/calculation and proving their validity/correctness.
- Examples: To compute $a + b$ ($a, b \in \mathbb{N}_0$), while $a > 0$ and $b > 0$:
  - v1: increment either a or b, and decrement the other.
  - v2: increment a and decrement b.
  - v3: if $a < b$, swap a and b; increment a and decrement b.

  Finally, rewrite $a + 0$ to a and $0 + b$ to b.



(An LLM should be able to translate v3 into actual (e.g.) Python code.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

Calculation, computation, algorithm

- One part of mathematics is deriving useful rules for computation/calculation and proving their validity/correctness.
- Example: In fact, incrementing and decrementing a decimal number is an algorithm in the original sense. (See the Python introduction.)

$$38998_1 \to 38999$$
$$38999_1 \to 3899_10 \to 389_100 \to 38_1000 \to 39000$$

(This iterated rewriting is what your computer does, but it only uses 0s and 1s. This idea is als formalized in what we call "Turing machine" and "semi-Thue system".)

- "Algorithm" originates from "Algoritmi" which is the Latinized name of Al-Khwarizmi who wrote several textbooks which were highly influential for modern mathematics:
  - *Kitāb al-Mukhtaṣar fī Ḥisāb al-Jabr wal-Muqābalah* gave "recipes" for solving special cases of quadratic equations ("algebra" goes back to "al-Jabr").
  - *Kitāb al-ḥisāb al-hindī* and *kitāb al-jam' wa'l-tafrīq al-ḥisāb al-hindī* gave "recipes" for arithmetic, in particular how to compute with numbers represented as "words over the alphabet of Hindu-Arabic digits". Originally, "algorithms" were "recipes" for computing with numbers in decimal representation.

For personal use only, no commercial use. FAU CS1 Department of Computer Science

Calculation, computation, algorithm

- One part of mathematics is deriving useful rules for computation/calculation and proving their validity/correctness.
- "School math" often focuses on practicing (training) the application of rules for computing with numbers and "elementary functions" (e.g. derivatives and integrals), while mathematics is actually concerned with finding the rules and proving that they "make sense" (under certain assumptions).

$$
\begin{array}{lll}
(\sin(x)\cdot\exp(x^3))' & \to & (\sin(x))'\cdot\exp(x^3)+\sin(x)\cdot(\exp(x^3))' \quad \text{(product rule)} \\
 & \to & \cos(x)\cdot\exp(x^3)+\sin(x)\cdot(\exp(x^3))' \quad \text{(derivative of sin)} \\
 & \to & \cos(x)\cdot\exp(x^3)+\sin(x)\cdot\exp'(x^3)\cdot(x^3)' \quad \text{(chain rule)} \\
 & \to & \cos(x)\cdot\exp(x^3)+\sin(x)\cdot\exp(x^3)\cdot(x^3)' \quad \text{(derivative of exp)} \\
 & \to^+ & \cos(x)\cdot\exp(x^3)+\sin(x)\cdot\exp(x^3)\cdot(3x^2) \quad \text{(power rule)}
\end{array}
$$

("$\to$" can mean a lot of different, sometimes related, things; here it denotes the rewriting by means of rules with the last step actually a "big step" as it iterates the product rule.)

- The rules for computing the derivative are (essentially) deterministic, whereas those for the antiderivative are non-deterministic, making them "harder" to apply (requiring more experience, better search heuristics, "AI").

For personal use only, no commercial use – TUM CIT Department of Computer Science

Calculation, computation, algorithm

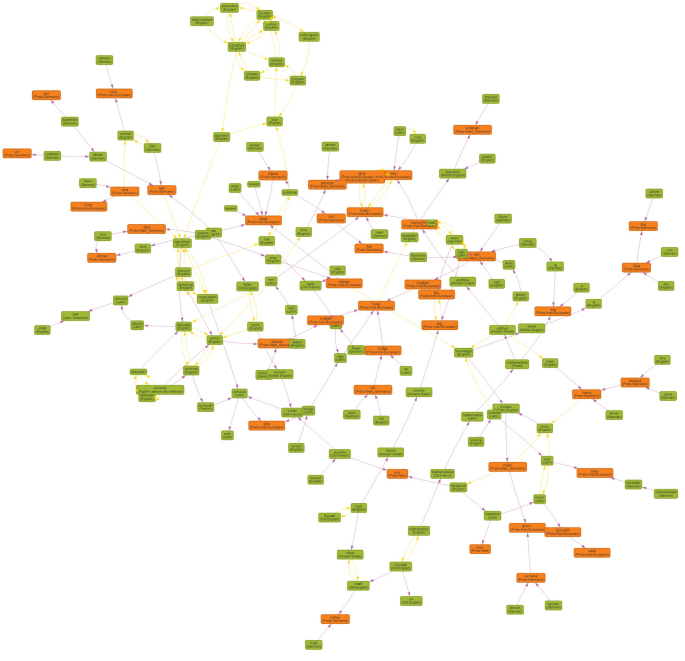
- One part of mathematics is deriving useful rules for computation/calculation and proving their validity/correctness.
- The main (and hardest) task of programming is finding the rules for a plan and arguing the correctness of the plan, i.e., why it will solve the given problem.

  The formulation of the plan in a "programming language" (the actual "coding") is just an intermediate step.

  (This is made precise in the Curry-Howard correspondence, which relates proofs, types, and computation i.e. it relates the foundations of logic, mathematics, and computer science (what Leibniz already had envisioned).)
- In the math part, we recall the basic mathematical concepts and corresponding "rules" you should know from school and the basic idea why they "make sense".

  But we will be more precise than what you are probably used to from school: both logic and mathematics are in the first place condensed fragments of language for precisely describing problems and reasoning about them.
- In the optional Python part, we have a look at how we can explain some of the most fundamental algorithms to the computer using Python (in order to recall even algorithms from elementary school ...).

For personal use only, no commercial use – TUM CIT Department of Computer Science



A bit of etymology:

- ▷ Green node: “actual word”
- ▷ Orange node: “reconstructed word”
- ▷ Yellow edge: “**can** have the meaning”
- ▷ Pink edge: “goes back to (etymology)”
- ⚠ Certainly incomplete and simplified (some paths have been contracted).
- ⚠ In logic and mathematics, the meaning of objects stems mostly from their interrelation (Leibniz, Herbrand, free interpretation, free algebraic structures, SCCs, semantic equivalence (“synonymous”), ...).
- ⚠ When training an LLM, “can have the meaning” is treated as “geometrically close”, and the computer tries to “learn” (rather “find”) a suitable distance measure by means of **a lot of** examples.
- ⚠ The graph represents some of the “knowledge” stored in `en.wikitionary`, but “knowledge” is not the same as “truth”.

For personal use only, no commercial use – TUM CIT Department of Computer Science

Basic meaning of “program” and “algorithm” and related words (*)

- Taken from wiktionary:
  - algorithm: 1. A collection of ordered steps that solve a mathematical problem. A precise step-by-step plan for a computational procedure [...]
  - program: 1. A set of structured activities; a plan of action.
  - plan: 2. A set of intended actions, usually mutually related, through which one expects to achieve a goal. 4. A method; a way of procedure; a custom.
  - procedure: 1. A particular method for performing a task. 2. A series of small tasks or steps taken to accomplish an end.
  - method: 1. A process by which a task is completed; a way of doing something.
  - process: 1. A series of events which produce a result (the product).
  - function: 6. A relation in which each element of the domain is associated with exactly one element of the codomain. 7. A routine [...]
  - routine: 1. A course of action to be followed regularly; a standard procedure. 2. A set of normal procedures, often performed mechanically.
  - code: 7. Instructions for a computer [...], 8. A program (in a programming language)
  - mechanical: 4. Done be a machine, 6. As if performed by a machine: lifeless, mindless, thoughtless, automatic (from Attic/Doric Greek “mekhane”/“makhana”).
  - “to calculate”: 2. To determine values or solutions by a mathematical process; reckon. 3. To plan; [...] (“calculus” (Latin): small limestone (chalk); cf. “calculus (logic/math)”.)
  - “to compute”: 1. To reckon, calculate.
  - “to reckon”: 1. To count; to enumerate; to number; also, to compute; to calculate. (cf. “regere”, “regula”, “rule”)

(Cf. also recipe (2,3), formula (3,1,8), formulation (1), ...)

For personal use only, no commercial use - TUM CIT Department of Computer Science

Greek alphabet and standard Latin abbreviations (*)

A, α	alpha	N, ν	nu
B, β	beta	Ξ, ξ	xi
Γ, γ	gamma	O, o	omikron
Δ, δ	delta	$\Pi, \pi/\varpi$	pi
$E, \epsilon/\varepsilon$	epsilon	$P, \rho/\varrho$	rho
Z, ζ	zeta	$\Sigma, \sigma/\varsigma$	sigma
H, η	(h)eta	T, τ	tau
$\Theta, \theta/\vartheta$	theta	Y, υ	upsilon (ypsilon)
I, ι	iota	$\Phi, \phi/\varphi$	phi
$K, \kappa/\varkappa$	kappa	X, χ	chi
Λ, λ	lambda	Ψ, ψ	psi
M, μ	mu	Ω, ω	omega

"i.e.": "that/it is", "this means" (Latin: "id est")
"e.g.": "for (the sake of an) example (instance)" (Latin: "exempli gratia")
"n.b.": "take good/special note" (Latin: "nota bene")
"cf.": "compare" (Latin: "confer")
"w.r.t.": "with respect to"

For personal use only, no commercial use – TUM CIT Department of Computer Science